USER MANUAL

Robot SDK — Python User Manual

Windows and Linux x86_64 | Python 3.9–3.13

SDK 1.5.0 · Document version: V1.4

SAFETY — Validate every motion in a clear workspace at a low speed setting. Software limits are an additional safeguard; they do not replace installation checks, emergency-stop procedures, or application-level interlocks.

Document revision history

Version	Date	Notes
V1.0	2026-07-23	Initial separate C++ and Python user manuals (Spark2 SDK).
V1.1	2026-08-06	C++17 application language. Windows tools restated for Visual Studio later than 2022 with MSVC v143. Python 3.10–3.14. Control priority. Path length rules. t in seconds.
V1.2	2026-08-10	Packaged install. Control priority (physical button above standard; same-tier wait). v=0 described as maximum speed. Gripper pos as 0–1. Limit overrun. State feedback rename.
V1.3	2026-09-24	Aligned to Robot SDK 1.3.0. Client is Robot; session is connect / disconnect (Python stop remains an alias of disconnect). No configuration path on the live client. Connect with RtConnectionInfo or an enumerated robot. Joint vectors follow the runtime joint count. v=0 keeps the current speed ratio and does not select maximum speed. moveVel / move_vel acceleration default is 10. Gripper command length is the gripper joint count. Added wait-until-idle, events, diagnostics, and network configuration. Public motion, feedback, and configuration target arm slot 0. The same manual covers the Windows x64 MSVC 2019 package (Python 3.9–3.12) and the Linux x86_64 GCC package (Python 3.9–3.13).
V1.4	2026-11-11	Aligned to Robot SDK 1.5.0 (Linux x86_64 package 2026-11-04, GCC 9.4). Per-arm and synchronized dual-arm handles for C++ and Python. MotionOptions for joint-point and home moves; the numeric v, t overloads of those calls are deprecated. Speed mode requires v in (0, 100]; v = 0 is rejected. t > 0 ignores v and uses full configured limits. Paths select quintic-path smoothing. Joint waits require arrival within 1 degree and report a rejected plan. Added fold, stop motion, safeguard stop, gripper wait, stop recording, and save trajectory. Connection status replaces connection metrics. Clear faults from the robot client. disconnect() may be called from an event callback.

Document scope

This manual describes how to install Robot SDK 1.5.0 (Python) and command the robot.

At a glance

Item	Supported / supplied
SDK	Robot SDK 1.5.0
Linux package	2026-11-04. Linux x86_64, GCC 9.4. Python wheels 3.9–3.13. examples/cpp and examples/python.
Windows package	Windows 10 / 11 x64, MSVC 2019 toolset v142. Python wheels 3.9–3.12. examples/cpp, examples/python, and examples/csharp.
Language	Python 3.9–3.13 on Linux; 3.9–3.12 on Windows
Build / install	pip install of the wheel in bindings/python for that OS
Document version	V1.4 (2026-11-11)

Contents

- Document revision history
- 1 Installation and first run
- 2 Runtime workflow and conventions
- 3 Core motion API
- 4 Interaction, feedback, and status
- 5 Configuration and safety limits
- 6 Kinematics and frame transforms
- Appendix A API index and types

Terminology

Term	Meaning
Joint space	A command or feedback whose length is joint_count() / jointCount() after connect. Positions are degrees. Velocities are degrees per second.
Arm slot 0	The arm addressed by Robot convenience methods. Use arm(index) for another arm. Use the dual-arm handle to move arms 0 and 1 together.
EE pose	Pose of the robot end-effector frame.
Tool pose	Pose of the configured tool frame. Same orientation as the EE pose; translation includes the active tool offset.
Tool offset	Translational offset (x, y, z) of the tool frame relative to the EE frame. Not a full pose.
PTP	Point-to-point motion. The controller plans a motion to the target pose.
Linear	Cartesian line motion. The commanded frame follows a straight Cartesian path.
RobotResult	Success or failure of a command. A successful non-blocking motion result means the command was submitted, not that the arm has arrived or that the plan was accepted.
MotionOptions	Explicit speed-limited or timed request for joint-point and home moves. Preferred over numeric v, t on those calls.

1. Installation and first run

1.1 Python prerequisites

Package	Requirement
Windows x64	Windows 10 / 11. Python 3.9, 3.10, 3.11, or 3.12. Wheel tag win_amd64, cpXY matching that interpreter.
Linux x86_64	Linux x86_64. Python 3.9, 3.10, 3.11, 3.12, or 3.13. Wheel tag linux_x86_64. The 1.5.0 library in this package was built with GCC 9.4.
Robot	The robot is on the same Ethernet network, or connect with its IP address.

1.2 Package layout

Use the package built for the operating system you run. Keep the layout intact.

```
<prefix>/
  bindings/python/      wheels for this OS
  docs/                 API notes and this manual
  examples/python/      examples and utils.py
  examples/cpp/         C++ examples
  include/              C++ headers
  lib/                  libraries and CMake package
```

Windows also has bin/ (robot_sdk.dll, robot_sdk_c.dll) and examples/csharp/. Linux libraries are lib/librobot_sdk.so and lib/librobot_sdk_c.so.

1.3 Quick start

Install the wheel whose tag matches the interpreter and the OS.

```
# Windows, Python 3.12
pip install .\bindings\python\robot_sdk-1.5.0-cp312-cp312-win_amd64.whl
# Linux, Python 3.12
pip install ./bindings/python/robot_sdk-1.5.0-cp312-cp312-linux_x86_64.whl
python -c "from robot_sdk import Robot; print('Robot SDK import ok')"
```

The robot's default IP address is 192.168.10.147. Connect the PC to the robot with an Ethernet cable. Set the PC's Ethernet address to 192.168.10.*, subnet mask 255.255.255.0. The last number is 1–254 and must not be 147.

Run an example from the package root. With no IP, it connects to the first robot found on the LAN. Pass the robot IP to select one robot. Public examples also include fold_arm.py, teach_and_playback.py, dual_arm_move_point.py, and dual_arm_move_path.py.

```
# Windows
cd .\examples\python
python .\move_point.py
python .\move_point.py 192.168.10.147
# Linux
cd examples/python
python3 move_point.py
python3 move_point.py 192.168.10.147
```

Minimal program:

```
from robot_sdk import Robot, RtConnectionInfo, enumerate_robots

robot = Robot()
result = robot.connect(RtConnectionInfo())  # auto-discover
if not result.ok:
    raise SystemExit(result.message)
mask = [True] * robot.joint_count()
robot.enable_arm_joint(mask).raise_for_error()
# move_pos, move_tool_point, go_home, ...
robot.disconnect()

robot.connect(RtConnectionInfo.to("192.168.10.147"))
robots = enumerate_robots()
if robots:
    robot.connect(robots[0])
```

enumerate_robots() is a module function. It does not keep a session. Pass one returned robot to connect().

1.4 First-run checklist

- Confirm the robot, controller, and safety system have finished their normal start-up checks.
- Connect, then read joint_count() and size every joint command to that length.
- Keep the arm clear of people, fixtures, cables, and end-of-travel positions.
- Start at a low speed, for example MotionOptions.speed_limited(20), and confirm the commanded frame before raising speed.
- Call disconnect() when the application exits.

2. Runtime workflow and conventions

2.1 Recommended control sequence

Step	Action	Why it matters
1	Robot()	Create the client.
2	connect(info or robot)	Starts the session. Empty server IP auto-discovers; a known IP or an enumerated robot skips discovery.
3	enable_arm_joint(mask)	Enables arm slot 0. The mask length must equal the joint count reported after connect.
4	Read status, then command motion	Confirm the arm is idle and enabled before increasing speed.
5	disconnect()	Ends the session and releases control. Do not rely on process exit.

2.2 Client lifecycle

Call	Parameters and result	Use / precondition
Robot()	No arguments.	Create the client once, then connect.
enumerate_robots()	Module function. Returns robots found on the network. Does not keep a session.	Use when more than one robot may answer, then connect to one entry.
connect(info or robot)	RtConnectionInfo, or one discovered robot. Returns RobotResult.	Required before motion, teach, playback, configuration writes, or live feedback. A normal connect waits for fresh Idle.
disconnect() / stop()	Ends the session. Returns RobotResult. stop() is the same operation.	Call on orderly shutdown. Allowed from an event callback; socket cleanup finishes on the next application-thread call.
enable_arm_joint(mask)	Boolean mask, length = joint count. true enables that joint.	Enables arm slot 0. For both arms, use dual_arm().enable_arm_joint(mask0, mask1).
arm(index)	Non-owning handle for one arm.	Reacquire after reconnect. Index is validated against topology after connect.
dual_arm()	Synchronized handle for arms 0 and 1.	Raises if the robot is not connected or has fewer than two arms.
set_arm_smoothing_method(method)	Smoothing strategy.	On Robot, applies to every arm. On arm(i), applies to that arm. Path calls select quintic-path themselves.
release_control()	No motion arguments. Returns RobotResult.	Drops the SDK control token. Later commands fail until a new connect.
stop_motion()	Optional wait_for_command_ack, default false. Holds the current measured pose.	Software stop while jogging or during a move. Does not enter ERROR and does not replace the hardware emergency stop.
trigger_safeguard_stop()	No arguments. Returns RobotResult.	Protective stop, same class as the panel abort. Latches until the robot reports ERROR. Requires connect; does not require holding control.

FAULTS — Read get_status() for diagnostics. Clear a fault from the robot client, then connect again if the session is no longer ready.

2.3 Control priority

The real-time controller decides which client may command the robot. Several clients can be present at once (physical teach button, Robot SDK, panel, web). A lower-priority client is blocked while a higher-priority client holds control. Clients in the same priority tier do not preempt each other: a same-tier client waits until the current controller finishes and releases control.

Priority (high to low)	Clients (examples)	Implication
Physical button	Physical teach button	Highest operator client. While it holds control, standard clients lose exclusive control.
Standard	Robot SDK; panel; web	Same tier. A standard client waits for the current standard controller to release control.

PRIORITY — If another client takes control, motion calls fail with CONTROL_LOST. Connect again after that client releases control.

2.4 Defaults and units

After connect, size every arm vector to that arm's joint count. Joint positions are degrees. Joint velocities are degrees per second. Gripper position commands are panel units in [0, 1] (0 open, 1 close), not the configurator soft or hard limits. Speed

v and acceleration a are percentages. Default v is 50. Default acceleration for a velocity command is 10. Time t is seconds. Default t is 0.

Robot convenience methods still address arm slot 0. arm_count() is the number of arms. gripper_count() is the number of gripper slots. A whole-robot idle wait covers every arm. A gripper-only move can leave the robot Idle, so wait on gripper feedback instead.

2.5 MotionOptions

Use MotionOptions for joint-point moves and home moves on Robot, on one arm, and on the dual-arm handle. speed_limited takes a percent in (0, 100] and leaves the duration to the controller. timed takes a positive duration in seconds and uses the full configured limits. A timed request that also lowers the speed cap returns Unsupported. The controller may extend a requested duration and will not shorten it.

The numeric (v, t) overloads of move_pos, move_pos_and_wait, go_home, and go_home_and_wait are deprecated on Robot, Arm, and DualArm. They remain callable and emit DeprecationWarning. t = 0 selects speed mode and requires v in (0, 100]. v = 0 is rejected and does not keep the previous ratio. t > 0 ignores v.

Path, Cartesian, velocity, gripper, and fold calls are not part of this deprecation. They still take numeric v and t. On a path, t[i] is the duration of that segment, not of the whole path. An empty time list selects speed mode for every segment. An empty speed list uses 50 percent on speed segments. A nonempty timing list must match the waypoint count. Mixed speed and time segments are allowed.

```
from robot_sdk import MotionOptions

# Preferred. Speed percent in (0, 100], duration chosen by the controller.
robot.go_home(options=MotionOptions.speed_limited(20)).raise_for_error()
robot.move_pos_and_wait(q, MotionOptions.timed(5.0), timeout_sec=30)

# Deprecated numeric form. Same rule: t = 0 uses v; t > 0 ignores v.
# robot.go_home(v=20, t=0.0)
```

2.6 Per-arm and dual-arm handles

Import Arm and DualArm from robot_sdk. dual_arm() raises RobotError when the robot is disconnected or has fewer than two arms. The handle keeps its Robot alive. Get a new handle after reconnect.

Both arms of one dual-arm call share the same timing. Each joint vector must match that arm's joint count. Path waypoint counts must match. Calling arm 0 and then arm 1 does not start a synchronized move. set_arm_smoothing_method on Robot still applies to every arm. Use the dual-arm home call when both arms must go home together.

```
left = robot.arm(0)
right = robot.arm(1)
both = robot.dual_arm()
both.enable_arm_joint(
    left.all_enabled_mask(), right.all_enabled_mask()
).raise_for_error()
both.move_pos_and_wait(
    q0, q1, MotionOptions.speed_limited(30)
).raise_for_error()
```

2.7 Completion, errors, and threads

A successful non-blocking motion call means the command was submitted on this machine. It does not mean the controller accepted the plan or that the arm has arrived. Use an and-wait call when the next step depends on arrival. A timeout does not cancel the motion. Call stop_motion() if the application must hold position when the wait expires.

Joint position, home, and fold waits succeed only after the arm is within 1 degree of the target and fresh consecutive Idle samples belong to this command. Dual-arm joint waits check both arms. If an updated controller clamps the target, the wait uses that effective target. If the effective target cannot be confirmed, the wait fails. An older controller keeps the requested target and can time out when it is not reached.

With an updated controller, a rejected plan is reported by the wait even if the robot state is Idle. An invalid target, an out-of-range measured start, or a profile that crosses a limit returns command-invalid. A boundary stop returns motion-interrupted. The message includes the command sequence and, when known, the arm and joint index starting at 0. These are motion errors, not a hardware fault. An older controller that does not send the rejection flags keeps the previous timeout behavior.

Path calls wait for the configuration acknowledgement and the command acknowledgement, then return. They do not wait until the path finishes. A configuration mismatch fails before the motion is sent. Check the result before starting a wait.

Result	What to do
Success on a non-blocking move	Command submitted. Wait, or read status, before assuming arrival.
Invalid argument	Fix the vector length, index, speed, or duration. Sending the same input again will fail the same way.
Command invalid	The target, measured start, or profile was rejected. Inspect the message. This is not by itself a hardware fault.
Motion interrupted	A boundary stop, or the wait was interrupted by recovery or the teach button.
Timeout	The wait expired. The motion is still active until you stop it.
Control lost	Another client holds control. Connect again after it releases control.
Unsupported	The call is not available. A timed move with a speed cap, or trajectory export on an older controller, returns this.

Call connection, motion, and query APIs for one Robot from one thread. State callbacks run on the SDK event thread and are serialized per Robot. The motion-done callback runs on the thread that called the wait. Keep handlers short. disconnect() from a callback closes the session immediately. Do not call connect() or a blocking motion API from a callback. Check result.ok or call raise_for_error(). dual_arm() can raise RobotError.

CALLBACK — Do not call connect, a blocking motion, or a blocking wait from a callback. disconnect is allowed.

3. Core motion API

Calls on Robot return RobotResult and command arm slot 0 unless noted. The same motion methods exist on arm(index) for one arm. Arm joint lists must match that arm's joint count.

3.1 Joint-space motion

Call	Parameters	Use / behavior
move_pos(arm_pos, options)	arm_pos: degrees, length = joint count. options: MotionOptions.	Preferred joint-point move for arm slot 0.
move_pos(arm_pos, v=50, t=0)	Deprecated numeric form.	t = 0 requires v in (0, 100]. t > 0 ignores v.
move_pos_path(path, v_path, t_path)	Joint waypoints in degrees. v and t are per waypoint.	Runs a joint path. The call selects quintic-path smoothing and waits for configuration and command acknowledgement.
move_vel(arm_vel, a=10, t=0)	arm_vel: deg/s. a: acceleration percent, default 10. t: seconds, default 0.	Command joint velocity. Not deprecated; numeric timing remains.
go_home(options)	MotionOptions.	Preferred move to the home pose. Confirm the path is clear.
go_home(v=50, t=0)	Deprecated numeric form.	Same speed and time rule as move_pos.
fold_arm(v=50, t=0)	Numeric v and t. Not a MotionOptions call.	Move arm slot 0 to the built-in fold posture.

3.2 Waiting for motion to finish

Call	Use
wait_until_idle(timeout_sec=30, poll_interval_sec=0.1)	Block until the whole robot is idle, a rejection is reported, or the timeout expires.
move_and_wait(move, timeout_sec=30, ...)	Send one motion callable, then wait.
move_pos_and_wait(arm_pos, options, ...)	Joint-point move, then arrival within 1 degree and fresh Idle.
go_home_and_wait(options, ...)	Home move, then the same arrival check.
fold_arm_and_wait(v=50, t=0, ...)	Fold, then the same arrival check. Numeric v and t.
move_gripper_pos_and_wait(pos, ...)	Gripper command, then wait on gripper feedback. Do not use idle-wait for a gripper-only move.

MOTION PATH — A successful API call does not prove that the swept path is safe. Validate fixture clearance, tool geometry, cable routing, and singularity or joint-limit behavior for the whole task.

3.3 Cartesian motion: EE frame vs. tool frame

Use an EE command when the target is defined on the end-effector frame. Use a tool command when the target is defined on the active tool frame so the configured tool offset is applied. These calls keep numeric v and t.

Target frame	PTP	Linear	Path variant	When to use
End effector	move_ee_point	move_ee_line	move_ee_point_path / move_ee_line_path	Poses are defined on the end-effector frame.
Tool	move_tool_point	move_tool_line	move_tool_point_path / move_tool_line_path	The task is defined on the configured tool frame.

3.4 Cartesian parameters

Parameter	Meaning	Practical note
pose	Target Cartesian pose in the selected frame.	Confirm the frame before commanding motion.
path	Ordered Cartesian waypoints in the selected frame.	Keep one frame for the whole path. A nonempty speed or time list must match the waypoint count.
v / v path	Speed percent for one target or each waypoint.	Default single-target v is 50. v = 100 is full scale. See section 2.5 for which calls reject v = 0.
t / t path	Time in seconds for one target or each waypoint.	Default t is 0. On a position-style timed move the controller may extend the duration and will not shorten it. A time-only path uses speed ratio 50 on a speed segment.

FRAME CHECK — Do not mix EE poses and tool poses. If the tool offset changes, revalidate task poses, approach paths, and saved waypoints.

3.5 Synchronized dual-arm motion

Obtain the handle with dual_arm() after connect on a robot whose arm_count() is at least 2.

Call	Use
enable_arm_joint(mask0, mask1)	Enable both arms in one configuration.
move_pos / move_pos_and_wait	Shared MotionOptions. Each vector matches that arm.
move_pos_path / move_pos_path_and_wait	Shared numeric v and t paths. Waypoint counts match.
move_ee_* / move_tool_*	Point, line, and path forms, with matching and-wait calls. Numeric v and t.
move_vel / move_vel_and_wait	Joint velocity on both arms. Acceleration percent, default 10.
go_home / go_home_and_wait	Shared MotionOptions. Both arms move home together.

See examples/python/dual_arm_move_point.py and dual_arm_move_path.py.

4. Interaction, feedback, and status

4.1 Gripper, teach, and playback

Call	Parameters / result	Use / precondition
move_gripper_pos(pos, v=50, t=0)	pos length is gripper_joint_count(). Values are [0, 1].	Command gripper joints. 0 is open and 1 is close.
move_gripper_pos_and_wait(...)	Same command, then gripper-feedback wait.	Use this when the next step needs the gripper to finish. Idle-wait can return early.
start_teach()	No extra arguments. Returns RobotResult.	Enter drag teaching and start recording. A new start after a stop replaces the previous recording.
stop_recording()	No extra arguments. Returns RobotResult.	Freeze the recording and leave drag enabled. Does not write a file and is not pause or resume.
stop_teach()	No extra arguments. Returns RobotResult.	End teaching and command a joint-position hold. Acknowledgement means the hold was accepted, not that the arm is already still.
save_trajectory(local_path)	Client-side JSON path. Returns RobotResult.	Export the frozen recording. Call it only after stop_recording or stop_teach. The file is replaced only after a complete export.

Call	Parameters / result	Use / precondition
start_playback() / stop_playback() / reset_playback()	No extra arguments. Returns RobotResult.	Start, stop, or reset playback of the recording held in controller memory. The JSON export is not loaded back.

Robot teach and playback calls are the arm slot 0 forms. arm(i) exports that arm's frozen samples. Starting or stopping recording is shared by the controller across arms; each arm still needs its own stop-teach if both were in drag mode. The JSON file uses schema robot_sdk.teaching_trajectory, version 1. Joint samples are radians, not degrees, and do not include the gripper. There is no file import.

save trajectory returns motion-interrupted if recording is still running, command-invalid if there is no recording, unsupported if the controller has no teach export, and transport-unavailable if TCP is down.

```
robot.start_teach().raise_for_error()
robot.stop_recording().raise_for_error()   # drag remains
robot.stop_teach().raise_for_error()       # hold current joints
robot.save_trajectory("teach.json").raise_for_error()
robot.wait_until_idle().raise_for_error()
```

See examples/python/teach_and_playback.py. Optional --trajectory writes the JSON file.

4.2 State feedback

Feedback calls on Robot apply to arm slot 0 and are available after connect. The same reads exist on arm(index). Joint position is degrees. Joint velocity is degrees per second.

Call	Returns	Use
get_joint_state() / get_joint_vel() / get_joint_tor()	Arm slot 0 position, velocity, or torque.	Monitor or log joint feedback. Size follows the joint count.
joint_positions()	Arm slot 0 positions only, in degrees.	Shorter read when velocity and torque are not needed.
get_ee_pose() / get_tool_pose()	Current EE pose or tool pose.	Read the same frame you command.
get_arm_joint_enabled_mask()	Per-joint enable mask, length = joint count.	Confirm joints are enabled before motion.
get_status()	SystemStatus: robot state, plan result, and diagnostic flags.	Joint diagnostics are indexed by arm, then joint.
print_status(status)	Prints a SystemStatus value.	Print status while commissioning.

4.3 Events and connection health

Use get_status() for diagnostics. Connection health is the session liveness fields below.

Call	Thread / result	Use
set_on_state_changed(cb)	SDK event thread. Previous state, current state, and status.	Robot state changed. Keep the handler short.
set_on_motion_done(cb)	Thread that called the wait. Status and result.	Motion wait finished, including a rejection result.
clear_event_callbacks()	Clears the registered callbacks.	Call from the application thread when shutting handlers down.
is_connected()	True while the session has not latched a receive timeout.	Quick liveness check. Status reads do not run the watchdog.
get_connection_status()	connected, telemetry_timed_out, receive_age_ms, receive_timeout_ms (3000).	A timeout stays latched until the next successful connect.

CALLBACK — Do not call connect or a blocking motion API from a callback. disconnect is allowed and closes the session at once.

4.4 Network configuration

These calls require an active session. Changing the wired address drops the current session; connect again at the new address.

Call	Use / caution
get_network_config()	Read wired and Wi-Fi settings. Default acknowledgement timeout 5000 ms. Returns the result and the config. The Wi-Fi password is masked.
set_wifi(ssid, password, enable=True)	Connect robot Wi-Fi to a router.
set_eth_static(ip, netmask, gateway, dns="")	Set a static wired address. The link drops. Reconnect with the new IP.
reset_network_config()	Restore factory network settings. Expect the current session to end.

5. Configuration and safety limits

Obtain the configuration service with get_configurator() after connect. On Robot it applies to arm slot 0. These limits are software safeguards. Review them together with mechanical stops, the controller hardware limits, and the cell safety design. Setter calls return RobotResult and wait for acknowledgement.

Call	Parameters / result	Use / caution
get_configurator()	Returns the Configurator.	Valid for the connected session. Arm slot 0. arm(i).get_configurator() addresses that arm.
set_tool_offset(offset) / get_tool_offset()	offset: Position (x, y, z), not a full pose.	Set and read back before tool-frame motion. Changing it changes the meaning of tool targets.
set_arm_pos_limits(limit)	Per-joint position software limit. Length follows the joint count.	Set only after checking the robot's reachable range. Commands past this limit are held at the limit.
set_arm_vel_limits(limit)	Per-joint velocity software limit.	Set only after checking the robot's speed capability.
get_arm_pos_limits() / get_arm_vel_limits()	Active software limits.	Read back during commissioning and before a task.
get_arm_max_pos()	Hardware maximum position.	Crossing this limit faults the robot. Recovery is done from the robot client.
get_arm_max_vel()	Hardware maximum velocity.	The highest velocity the hardware allows.
get_arm_max_pos_jump() / get_arm_max_vel_jump()	Anti-jump thresholds for position and velocity.	A step larger than the threshold is saturated at the jump limit.
get_arm_max_pos_follow_error()	Position following-error threshold.	Crossing it faults the robot.
get_arm_max_vel_follow_error() / get_arm_max_tor_follow_error()	Velocity and torque following-error thresholds.	Read them for monitoring.
set/get_gripper_pos_limits(), get_gripper_max_pos()	Gripper software limits and hardware maximum.	These are safety limits. They are not the [0, 1] units of move_gripper_pos.

6. Kinematics and frame transforms

Kinematics is a separate class. It does not command motion. It needs a configuration directory that contains arm_v2.urdf and meshes/. Joint vectors use degrees and must match the joint count of that kinematics object. The live Robot class does not load this directory.

> **THIS PACKAGE** — This release does not include the kinematics library or a configuration directory.

Call	Input / output	Use
Kinematics(config_prefix_path, model_id="")	Configuration directory, optional model id.	Load the URDF model. Default model is spark2_v2.
forward_kinematics(arm_pos)	Input: joint positions in degrees. Output: EE pose.	Pose for a known joint state.
inverse_kinematics(ee_pose, arm_pos)	Input: EE pose and a seed joint vector. Output: RobotResult and solved joints.	Use the solved joints only when the result is success.
ee_to_tool_pose(ee_pose, tool_offset)	EE pose plus a translational tool offset.	Planning or logging in the tool frame.
tool_to_ee_pose(tool_pose, tool_offset)	Tool pose plus a translational tool offset.	Express a tool target on the EE frame.
euler_to_quaternion(euler)	Intrinsic XYZ Euler angles to quaternion.	Build a Pose orientation.
quaternion_to_euler(quat)	Quaternion to intrinsic XYZ Euler angles.	Display or log orientation.

IK — An inverse-kinematics solution is only a kinematic result. Check joint limits, collisions, singularity margin, and the approach path before passing it to a motion command.

Appendix A. API index and types

A.1 API groups

Group	Operations
Lifecycle	Robot, enumerate_robots, connect, disconnect, stop, enable_arm_joint, arm, dual_arm, release_control, stop_motion, trigger_safeguard_stop
Joint motion	move_pos, move_pos_path, move_vel, go_home, fold_arm, MotionOptions, wait_until_idle, move_and_wait, and the matching and-wait calls
Cartesian	move_ee_point, move_ee_line, move_tool_point, move_tool_line, and the matching path calls, on Robot, Arm, and DualArm
Interaction	move_gripper_pos, move_gripper_pos_and_wait, teach, stop_recording, save_trajectory, playback, feedback, status, events, connection status, network configuration
Configuration	get_configurator, tool offset, arm and gripper limits, hardware maximums, jump and following-error thresholds
Kinematics	FK, IK, Euler/quaternion, EE/tool transforms. Not included in this release.

A.2 Types

Type family	Meaning
RtConnectionInfo	server_ip only. An empty address auto-discovers. RtConnectionInfo.to(ip) sets a known host.
DiscoveredRobot	One row from enumerate_robots(): server address and robot name. Pass it to connect().
MotionOptions	speed_limited(percent) or timed(seconds). Percent is an integer in (0, 100]. Timed seconds must be greater than 0.
JointValues / JointMask	Runtime-length arm or gripper vectors and enable masks. Build with the joint count after connect.
JointState	Arm vector plus gripper vector for position, velocity, or torque feedback.
Pose / Position / Quaternion	Cartesian pose. Tool offset uses Position only.
RobotResult	ok, code, message. Check ok, or call raise_for_error(), before the next command.
SmoothingMethod	LINEAR, COS, CUBIC, QUINTIC, QUINTIC_PATH, LOW_PASS_FILTER, NONE. Path calls select QUINTIC_PATH. A low-pass setting on an ordinary position move runs as a finite cosine profile.
RobotState	STARTUP, IDLE, MOVING, SETTLING, ERROR, RECOVERY, SHUTDOWN.
PlanResult	SUCCESS, POSE_NOT_REACHABLE, LINEAR_PATH_FAILED, NONE. NONE is not a motion failure.
SystemStatus	robot_state, plan_result, robot and per-joint diagnostic flags.
ConnectionStatus	connected, telemetry_timed_out, receive_age_ms, receive_timeout_ms. Timeout is 3000 ms and stays latched until reconnect.